Integer Linear Programming (ILP)

Zdeněk Hanzálek
zdenek.hanzalek@cvut.cz

CTU in Prague

October 8, 2026

Table of contents

1. Introduction
   - Problem Statement
   - Comparison of ILP and LP
   - Examples
2. Algorithms
   - Enumerative Methods
   - Branch and Bound Method
   - Special Cases of ILP
3. Problem Formulation Using ILP
4. Conclusion

Problem Statement

Integer Linear Programming (ILP)

The ILP problem is given by matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and vectors $\mathbf{b} \in \mathbb{R}^m$ and $\mathbf{c} \in \mathbb{R}^n$. The goal is to find a vector $\mathbf{x} \in \mathbb{Z}^n$ such that $\mathbf{A} \cdot x \leq b$ and $c^T \cdot x$ is the maximum.

Usually, the problem is given as $\max \left\{ c^T \cdot x : \mathbf{A} \cdot x \leq b, x \in \mathbb{Z}^n \right\}$.

- A large number of practical optimization problems can be modeled and solved using Integer Linear Programming - ILP.

Comparison of ILP and LP

- The LP problem solution space is convex, since $x \in \mathbb{R}^n$
- The ILP problem differs from the LP problem in allowing integer-valued variables. If some variables can contain real numbers, the problem is called Mixed Integer Programming - MIP. Often MIP is also called ILP, and **we will use the term ILP when at least one variable has integer domain**.
- If we solve the ILP problem by an LP algorithm and then **just round the solution**, we could not only get the suboptimal solution, we can also obtain a solution which is not feasible.
- While the LP is solvable in polynomial time, **ILP is NP-hard**, i.e. there is no known algorithm which can solve it in polynomial time.
- Since the **ILP solution space is not a convex set**, we cannot use convex optimization techniques.

Example ILP1a: 2-Partition Problem

2-Partition Problem

- **Instance:** Number of banknotes $n \in \mathbb{Z}^+$ and their values $p_1, \ldots, p_n$, where $p_{i \in 1..n} \in \mathbb{Z}^+$.
- **Decision:** Is there a subset $S \subseteq \{1, \ldots, n\}$ such that $\sum_{i \in S} p_i = \sum_{i \notin S} p_i$?

The decision problem, which can be written while using the equation above as an ILP constraint (but we write it differently).

- $x_i = 1$ iff $i \in S$

This is one of the "easiest" NP-complete problems.

min	0
subject to:	
	$\sum_{i \in 1..n} x_i * p_i = 0.5 * \sum_{i \in 1..n} p_i$
parameters:	$n \in \mathbb{Z}^+$, $p_{i \in 1..n} \in \mathbb{Z}^+$
variables:	$\mathbf{x_{i \in 1..n}} \in \{\mathbf{0}, \mathbf{1}\}$

Example ILP1b: Fractional Variant of the 2-Part. Prob.

We allow division of banknotes such that $x_{i \in 1..n} \in \langle 0, 1 \rangle$. The solution space is a convex set - the problem can be formulated by LP:

$$
\begin{array}{lll}
\text{min} & 0 & \\
\text{subject to:} & & \\
& \sum_{i \in 1..n} x_i * p_i & = 0.5 * \sum_{i \in 1..n} p_i \\
& \mathbf{x_i} & \leq \mathbf{1} \quad i \in 1..n \\
\text{parameters:} & n \in \mathbb{Z}^+, \; p_{i \in 1..n} \in \mathbb{Z}^+ & \\
\text{variables:} & \mathbf{x_{i \in 1..n}} \in \mathbb{R}^+_{\mathbf{0}} &
\end{array}
$$

- For example: $p = [100, 50, 50, 50, 20, 20, 10, 10]$ the fractional variant allows for $x = [0, 0, 0.9, 1, 1, 1, 1, 1]$ and thus divides the banknotes into equal halves $100 + 50 + 5 = 45 + 50 + 20 + 20 + 10 + 10$, but this instance does not have a non-fractional solution.
- For some non-fractional instances we can easily find that they cannot be partitioned (e.g. when the sum of all values divided by the greatest common divisor is not an even number), however we do not know any alg that can do it in polynomial time for any non-fractional instance.

Example ILP1c: 2-Partition Prob. - Optimization Version

- The decision problem can be solved by an optimization algorithm while using a **threshold** value (here $0.5 * \sum_{i \in 1..n} p_i$) and comparing the optimal solution with the threshold.
- Moreover, when the decision problem has no solution, the optimization algorithm returns a value that is closest to the threshold.

$$
\begin{array}{ll}
\text{min} & C_{max} \\
\text{subject to:} & \\
 & \sum_{i \in 1..n} x_i * p_i \leq C_{max} \\
 & \sum_{i \in 1..n} (1 - x_i) * p_i \leq C_{max} \\
\text{parameters:} & n \in \mathbb{Z}^+,\ p_{i \in 1..n} \in \mathbb{Z}^+ \\
\text{variables:} & x_{i \in 1..n} \in \{0, 1\},\ C_{max} \in \mathbb{R}^+
\end{array}
$$

Application: the scheduling of nonpreemptive tasks $\{T_1, T_2, ..., T_n\}$ with processing times $[p_1, p_2, ..., p_n]$ on two parallel identical processors and minimization of the completion time of the last task (i.e. maximum completion time C_{max}) - $P2 \,||\, C_{max}$. The fractional variant of 2-partition problem corresponds to the preemptive scheduling problem.

Example ILP2a: Shortest Paths

Shortest Path in directed graph

- **Instance:** digraph G with n nodes, distance matrix $c : V \times V \to \mathbb{R}^+$ and two nodes $v_1, v_n \in V$.
- **Goal:** find the shortest path from v_1 to v_n or decide that v_n is unreachable from v_1.

LP formulation using a physical analogy:

- node = ball
- edge = string (we consider a symmetric distance matrix c)
- node v_1 is fixed, other nodes are pulled by gravity
- tightened string = shortest path

Is it a polynomial problem?

$$
\begin{array}{ll}
\max & l_n \\
\text{subject to:} & \\
& l_1 = 0 \\
& l_j \leq l_i + c_{i,j} \quad i \in 1..n, j \in 1..n \\
\text{parameters:} & n \in \mathbb{Z}^+,\ c_{i \in 1..n, j \in 1..n} \in \mathbb{R}^+ \\
\text{variables:} & l_{i \in 1..n} \in \mathbb{R}^+
\end{array}
$$

Example ILP3: Traveling Salesman Problem

Asymmetric Traveling Salesman Problem

- **Instance:** complete digraph K_n, distance matrix $c : V \times V \to \mathbb{Q}^+$.
- **Goal:** find the shortest Hamiltonian cycle, i.e. a subgraph with all n vertices and n edges such that the sequence $v_{[1]}, e_{[1]}, \ldots, v_{[n]}, e_{[n]}, v_{[1]}$ is a closed directed walk (tah) with $v_i \neq v_j$ for $1 \leq i < j \leq n$.

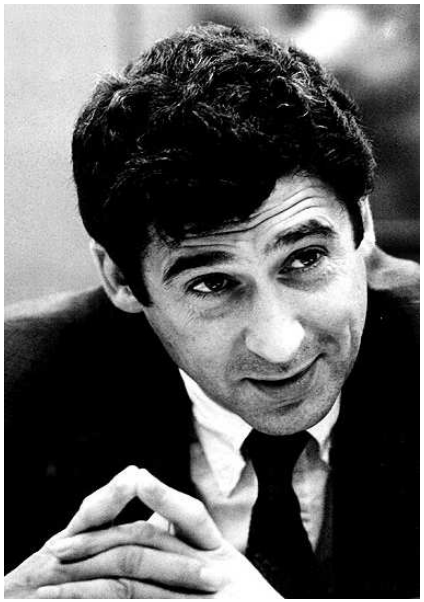
$x_{i,j} = 1$ iff node i is in the cycle just before node j
The enter and leave constraints do not capture the TSP completely, since any disjoint cycle (i.e. consisting of several sub-tours) will satisfy them.
We use s_i, the "time" of entering node i, to **eliminate the sub-tours**.

$$
\begin{array}{lrll}
\text{min} & \sum_{i\in 1..n}\sum_{j\in 1..n} c_{i,j} * x_{i,j} & & \\
\text{subject to:} & x_{i,i} = 0 & i \in 1..n & \text{avoid self-loop} \\
& \sum_{i\in 1..n} x_{i,j} = 1 & j \in 1..n & \text{enter once} \\
& \sum_{j\in 1..n} x_{i,j} = 1 & i \in 1..n & \text{leave once} \\
& s_i + c_{i,j} - (1 - x_{i,j}) * M \leq s_j & i \in 1..n, j \in 2..n & \text{cycle indivisibility} \\
\text{parameters:} & M \in \mathbb{Z}^+,\ n \in \mathbb{Z}^+,\ c_{i\in 1..n, j\in 1..n} \in \mathbb{Q}^+ & & \\
\text{variables:} & x_{i\in 1..n, j\in 1..n} \in \{0,1\},\ s_{i\in 1..n} \in \mathbb{R}_0^+ & &
\end{array}
$$

Algorithms

The most successful methods to solve the ILP problem are:

- Enumerative Methods
- Branch and Bounds
- Cutting Planes Methods





Ralph Gomory and Vašek Chvátal are prominent personalities in the field of ILP. Some of the solution methods are called: Gomory's Cuts or Chvátal-Gomory's cuts.

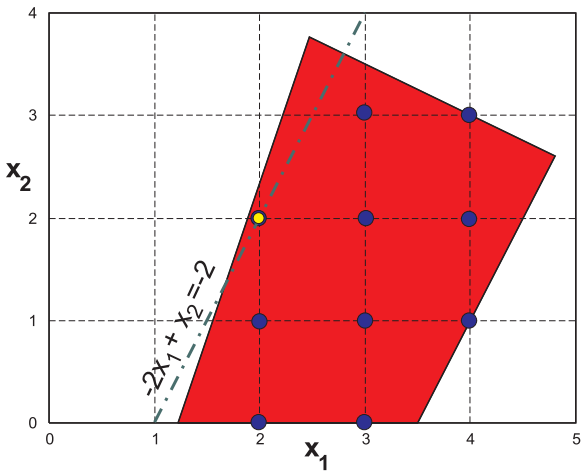
Enumerative Methods

- Based on the idea of **inspecting all possible solutions**.
- Due to the integer nature of the variables, the number of solutions is countable, but their number is huge. So this method is usually suited only for smaller instances with a small number of variables.
- The LP problem is solved for every combination of discrete variables.

Enumerative Methods

$$
\begin{array}{lrl}
\max & -2x_1 & + \; x_2 \\
s.t. & 9x_1 & - \; 3x_2 \geq 11 \\
 & x_1 & + \; 2x_2 \leq 10 \\
 & 2x_1 & - \; x_2 \leq 7 \\
 & x_1, x_2 \geq 0, & x_1, x_2 \in \mathbb{Z}_0^+
\end{array}
$$

The figure below shows 10 feasible solutions. The optimal solution is $x_1 = 2, x_2 = 2$ with an objective function value of -2.



Branch and Bound Method

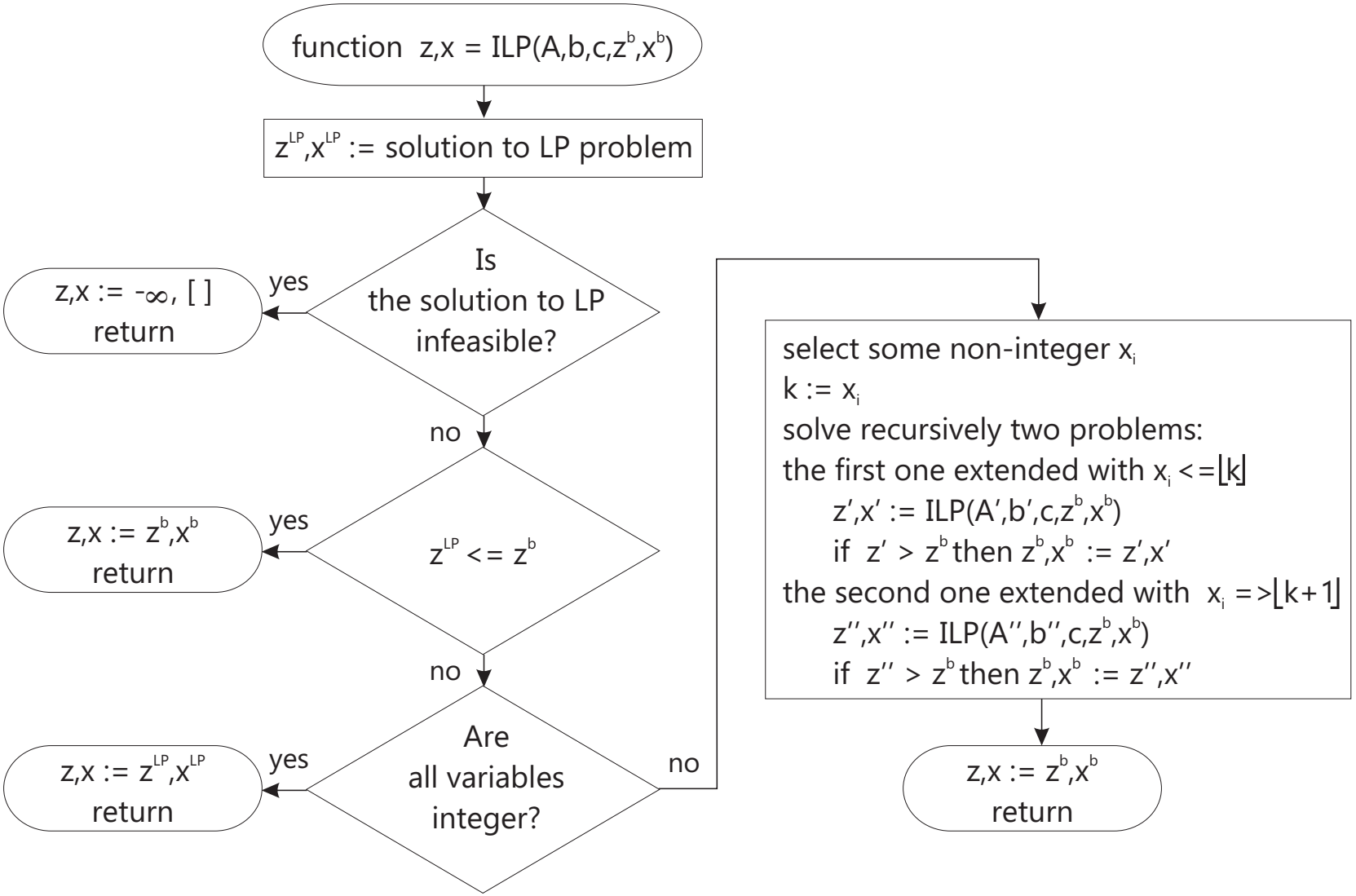
- The method is based on **splitting** the solution space into disjoint sets.
- It starts by relaxing on the integrality of the variables and **solves the LP problem**.
- If all variables x_i are **integers, the computation in this branch ends**. Otherwise one variable $x_i \notin \mathbb{Z}$ is chosen and its value is assigned to k.
- Then the solution space is **divided into two sets** - in the first one we consider $x_i \leq \lfloor k \rfloor$ and in the second one $x_i \geq \lfloor k \rfloor + 1$.
- The algorithm **recursively repeats** computation for the both new sets till feasible integer solution is found.

Branch and Bound Method

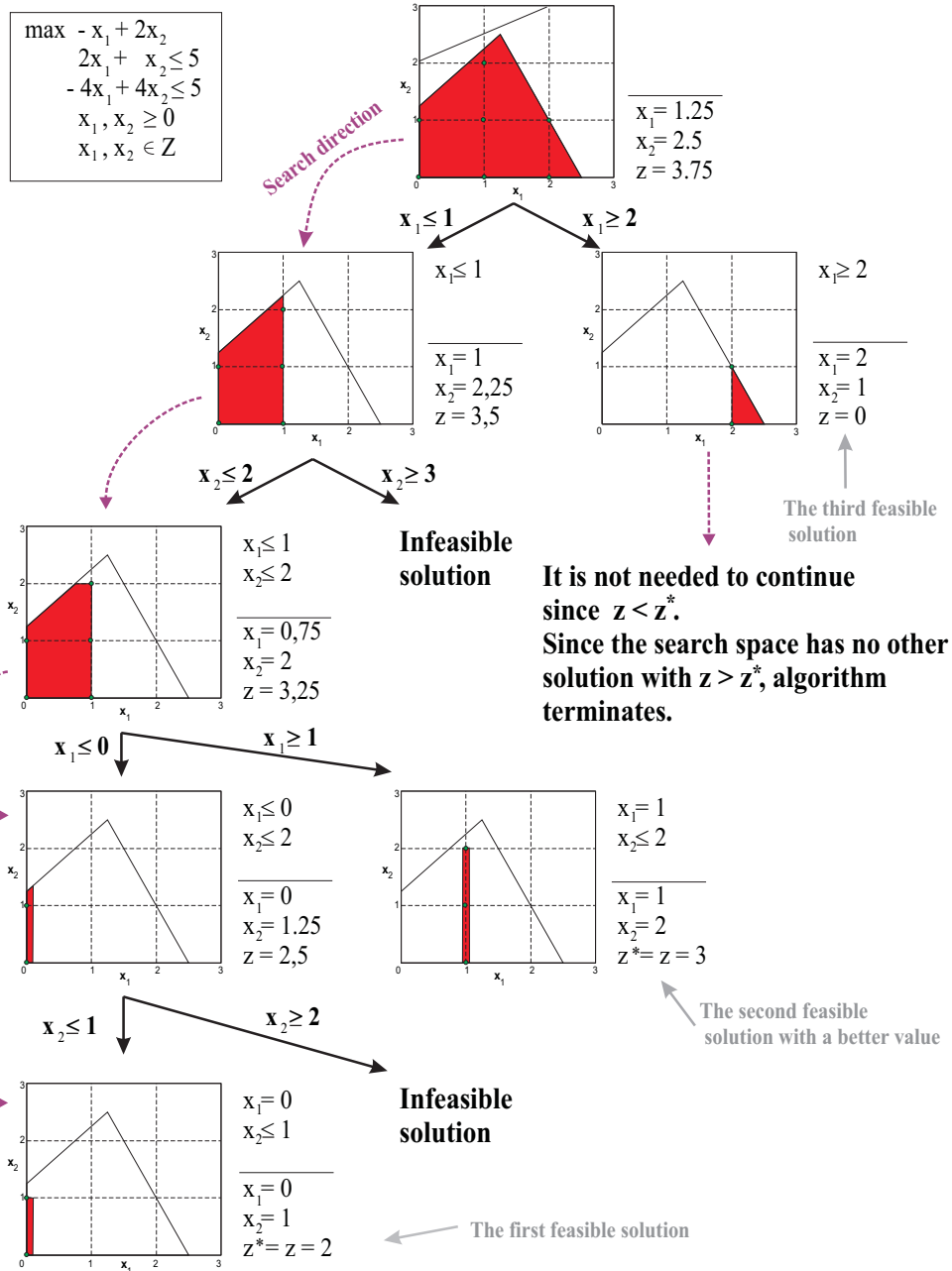
- By *branching* the algorithm creates a solution space which can be depicted as a **tree**.
- A node represents the **partial solution**.
- A **leaf** determines some (integer) solution or "bounded" branch (infeasible solution or the solution which does not give a better value than the best solution found up to now)
- As soon as the algorithm finds an integer solution, its objective function value can be used for *bounding*
  - The **node is discarded** whenever z, its (integer or real) objective function value, is worse than z^*, the value of the best known solution

The ILP algorithm often uses an LP **simplex method** because after adding a new constraint it is not needed to start the algorithm again, but it allows one to continue the previous LP computation while solving the dual simplex method.

Branch and Bound Algorithm - ILP maximization problem



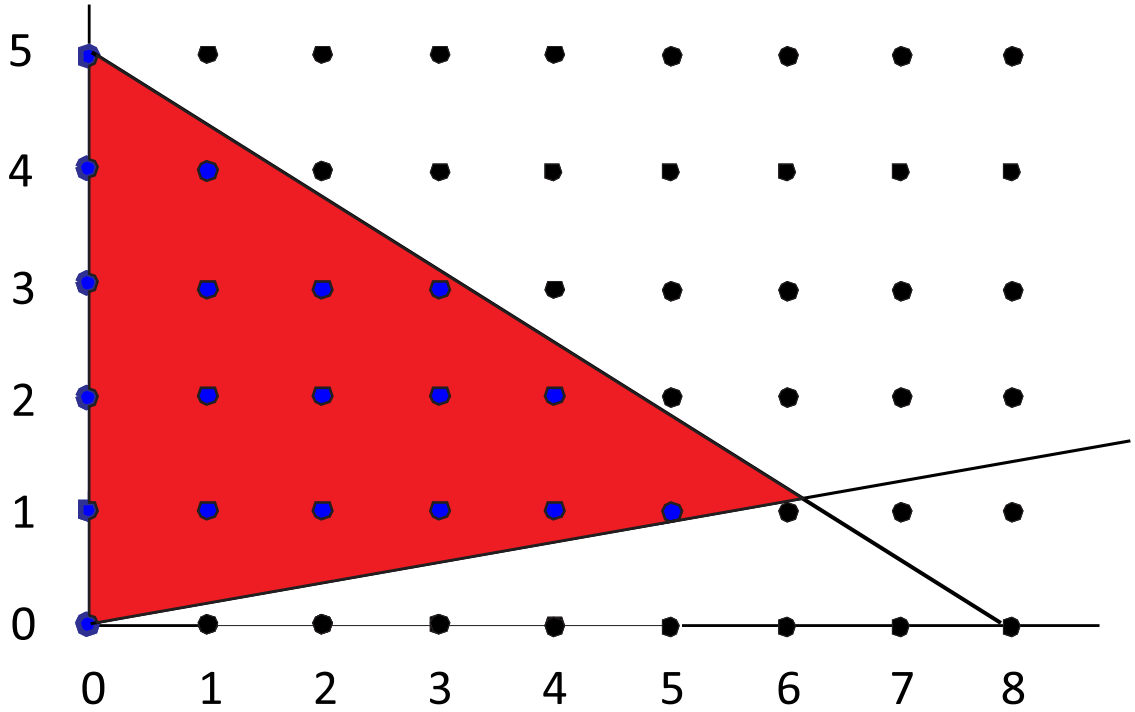
Branch and Bound - Example



ILP Solution Space

$$
\begin{aligned}
\max z = \quad & 3x_1 & + 4x_2 & \\
s.t. \quad & 5x_1 & + 8x_2 & \leq 40 \\
& x_1 & - 5x_2 & \leq 0 \\
& & x_1, x_2 & \in \mathbb{Z}_0^+
\end{aligned}
$$

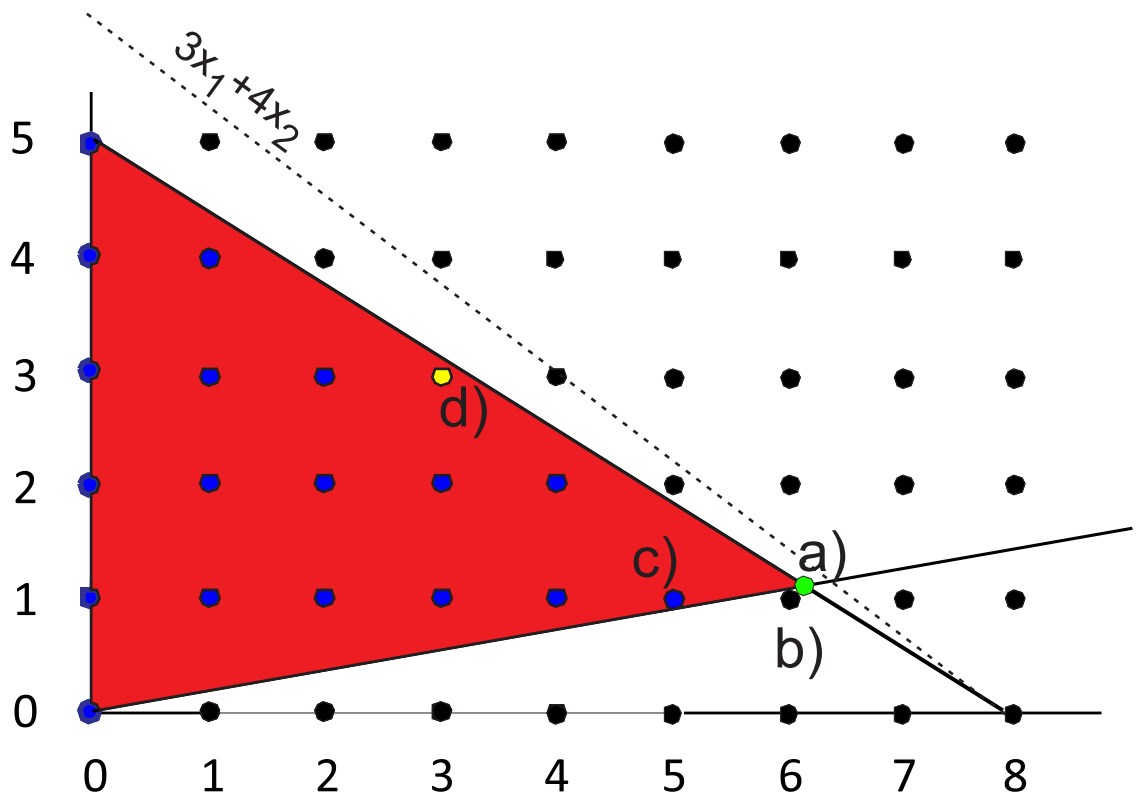
- What is optimal solution?
- Can we use LP to solve the ILP problem?



Rounding is not always good choice

$$\max z = \quad 3x_1 \quad + 4x_2$$

- a) LP solution $z = 23.03$ for $x_1 = 6.06, x_2 = 1.21$
- b) Rounding leads to infeasible solution $x_1 = 6, x_2 = 1$
- c) Nearest feasible integer is not optimal $z = 19$ for $x_1 = 5, x_2 = 1$
- d) Optimal solution is $z = 21$ for $x_1 = 3, x_2 = 3$



Why integer programming?

Advantages of using integer variables

- more realistic (it does not make sense to produce 4.3 cars)
- flexible - e.g. binary variable can be used to model the decision (logical expression)
- we can formulate NP-hard problems

Drawbacks

- harder to create a model
- usually suited to solve the problems with less than 1000 integer variables

Special Cases of ILP - Example ILP2b: Shortest Paths

Shortest path in a graph

- **Instance:** digraph G without a negative cycle given by incidence matrix $W : V \times E \to \{-1, 0, +1\}$ (such that $w_{ij} = +1$ when edge e_j leaves vertex i and $w_{kj} = -1$ when edge e_j enters vertex k), distance vector $c \in \mathbb{R}^+$ and two nodes $s, t \in V$.
- **Goal:** find the shortest path from s to t or decide that t is unreachable from s.

LP formulation:

- $x_j = 1$ iff edge j is chosen
- We enter the node as many times as we leave it (except s and t)
- Due to the minimization we can omit $x_j \leq 1$

$$
\begin{array}{lll}
\text{min} & \sum_{j \in 1..m} c_j * x_j & \\
\text{subject to:} & & \\
\sum_{j \in 1..m} w_{s,j} * x_j & = 1 & \text{source } s \\
\sum_{j \in 1..m} w_{i,j} * x_j & = 0 & i \in V \setminus \{s, t\} \\
\sum_{j \in 1..m} w_{t,j} * x_j & = -1 & \text{sink } t \\
\text{pars:} & w_{i \in 1..n, j \in 1..m} \in \{-1, 0, 1\},\ c_{j \in 1..m} \in \mathbb{R}^+ & \\
\text{vars:} & x_{j \in 1..m} \in \mathbb{R}_0^+ &
\end{array}
$$

The returned values of x_j are integers (binary) even though it is LP. Why?

Totally Unimodular Matrix leads to Integral Polyhedron

Polynomial time algorithm for general ILP is not known, however there are special cases which can be solved in polynomial time.

Definition - Totally unimodular matrix

Matrix $\mathbf{A} = [a_{ij}]$ of size m/n is totally unimodular if the determinant of every square submatrix of matrix $\mathbf{A}$ is equal 0, $+1$ or -1.

Necessary condition: if $\mathbf{A}$ is totally unimodular then $a_{ij} \in \{0, 1, -1\}\ \forall i, j$.

Lemma - Integral Polyhedron

Let A be a totally unimodular m/n matrix and let $b \in \mathbb{Z}^m$. Then each vertex of the polyhedron $P := \{x\,;\, \mathbf{A}x \leq b\}$ is an integer vector.

Proof: [Schrijver] Theorem 8.1.

Integer Solution by Polynomial Algorithm

Lemma - Integer solution by simplex algorithm

If the ILP problem is given by a totally unimodular matrix $\mathbf{A}$ and integer vector b then every feasible solution by a simplex algorithm is an integer vector.

Proof: From the Lemma on previous slide - the simplex algorithm inspects vertices that are integer vectors.
Unfortunately, the simplex algorithm does not have polynomial complexity.

Fortunately, there are polynomial algorithms able to solve the LP problems and to find the vertex in the facet with optimal solutions. This subject is studied in Linear Programming and Polyhedral Computation.

1st Sufficient Condition for Totally Unimodular Matrix

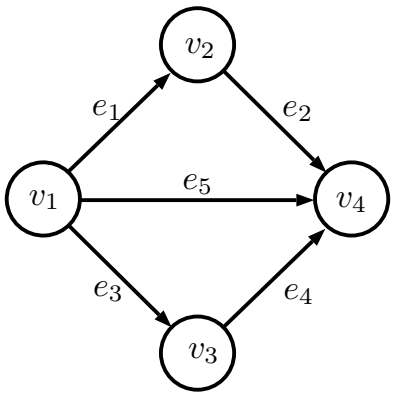
Lemma - Sufficient Condition - No more than one $+1$ and no more than one -1 in each column

Let **A** be matrix of size m/n such that

1. $a_{ij} \in \{0, 1, -1\}$, $i = 1, ..., m$, $j = 1, ..., n$
2. each column in **A** contains one non-zero element or exactly two non-zero elements $+1$ and -1

Then the matrix **A** is totally unimodular.

Proof: [Ahuja] Theorem 11.12. [KorteVygen] Theorem 5.26.
Example: ILP constraints of the Shortest Paths problem are: $W * x = b$



$$W = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} e_1 \quad e_2 \quad e_3 \quad e_4 \quad e_5 \\ \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \\ -1 & 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 1 & 0 \\ 0 & -1 & 0 & -1 & -1 \end{bmatrix} \end{array} \quad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix} \quad b = \begin{bmatrix} 1 \\ 0 \\ 0 \\ -1 \end{bmatrix}$$

2nd Sufficient Condition for Totally Unimodular Matrix

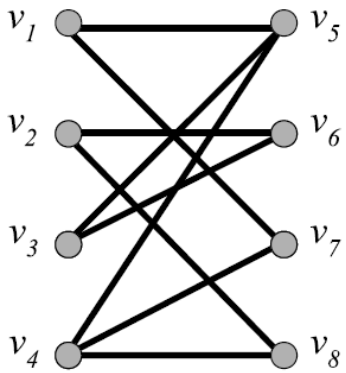
Lemma - Sufficient Condition - Partitioned Rows

Let **A** be matrix of size m/n such that

1. $a_{ij} \in \{0, 1\}$, $i = 1, ..., m$, $j = 1, ..., n$
2. each column in **A** contains at most two entries of value 1
3. the rows of **A** can be partitioned into two sets $\mathcal{V}_1$ and $\mathcal{V}_2$ such that every column of **A** has at most one entry of value 1 in the rows of $\mathcal{V}_1$ and at most one entry of value 1 in the rows of $\mathcal{V}_2$.

Then the matrix **A** is totally unimodular.

Example: bipartite graph related to assignment problem (lecture on Flows)



	v_1v_5	v_1v_7	v_2v_6	v_2v_8	v_3v_5	v_3v_6	v_4v_5	v_4v_7	v_4v_8
v_1	1	1	0	0	0	0	0	0	0
v_2	0	0	1	1	0	0	0	0	0
v_3	0	0	0	0	1	1	0	0	0
v_4	0	0	0	0	0	0	1	1	1
v_5	1	0	0	0	1	0	1	0	0
v_6	0	0	1	0	0	1	0	0	0
v_7	0	1	0	0	0	0	0	1	0
v_8	0	0	0	1	0	0	0	0	1

$$\min \quad \sum_{ij \in E} c_{ij} x_{ij}$$

$$\text{subject to:} \quad \sum_{j \in V_2 : \{i,j\} \in E} x_{ij} = 1, \quad \forall i \in V_1$$

$$\sum_{i \in V_1 : \{i,j\} \in E} x_{ij} = 1, \quad \forall j \in V_2$$

$$\text{pars:} \quad c_{ij} \in \mathbb{R}^+ \quad \forall \{i,j\} \in E$$

$$\text{vars:} \quad x_{ij} \in \{0,1\} \quad \forall \{i,j\} \in E$$

3rd Sufficient Condition for Totally Unimodular Matrix

Definition - Consecutive-Ones (a.k.a. Interval Matrix)

Matrix **A** of size m/n has the consecutive-ones property if

1. $a_{ij} \in \{0, 1\}$, $i = 1, ..., m$, $j = 1, ..., n$
2. there exists a permutation of its rows such that for any column j, $a_{ij} = a_{kj} = 1$ with $i < k$ implies $a_{lj} = 1$ for $i < l < k$.

Lemma - Sufficient Condition - Consecutive-Ones

The matrix **A** with the consecutive-ones property is totally unimodular.

Examples:

$$\mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix} \qquad \mathbf{A} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \end{bmatrix}$$

Example of Consecutive Ones: Workforce Scheduling

Minimize the total cost of employees for four 6 hour blocks.

- Satisfy minimum staffing requirements in each of them: Morning - 4 employees, Afternoon - 7, Evening - 5, and Night - 2.
- Each shift covers **subsequent** blocks: (1) Morning+Afternoon, (2) Afternoon+Evening, (3) Evening+Night, (4) Night, (5) All Day.
- One employee on shift j costs c_j.

Let x_j be the number of employees assigned to shift j.

$$
\begin{array}{lll}
\text{min} & \sum_{j \in 1..5} c_j * x_j & \\
\text{subject to:} & & \\
1 * x_1 + 0 * x_2 + 0 * x_3 + 0 * x_4 + 1 * x_5 & \geq 4 & \text{Morning} \\
1 * x_1 + 1 * x_2 + 0 * x_3 + 0 * x_4 + 1 * x_5 & \geq 7 & \text{Afternoon} \\
0 * x_1 + 1 * x_2 + 1 * x_3 + 0 * x_4 + 1 * x_5 & \geq 5 & \text{Evening} \\
0 * x_1 + 0 * x_2 + 1 * x_3 + 1 * x_4 + 1 * x_5 & \geq 2 & \text{Night} \\
\text{pars:} \quad a_{i \in 1..4, j \in 1..5} \in \{0,1\},\ c_{j \in 1..5} \in \mathbb{R}^+ & & \\
\text{vars:} \quad x_{j \in 1..5} \in \mathbb{Z}_0^+ & &
\end{array}
$$

Analysis: In each column of **A**, the ones form a single contiguous interval. What happens when we introduce "split shifts" (breaking the consecutive ones property)?

Other Sufficient Condition for Totally Unimodular Matrix

Several operations preserve totally unimodularity. For instance, when $\mathbf{A}$ is totally unimodular matrix of size m/n , then

1. every matrix obtained from $\mathbf{A}$ by permuting some rows and/or columns is totally unimodular,
2. every matrix obtained from $\mathbf{A}$ by multiplying some rows and/or columns by -1 is totally unimodular,
3. $\mathbf{A}^T$ is totally unimodular,
4. the matrix $(\mathbf{A}, \mathbf{I})$ is totally unimodular (where $\mathbf{I}$ is the $m \times m$ identity matrix),
5. the matrix $\begin{pmatrix} \mathbf{A} \\ \mathbf{I} \end{pmatrix}$ is totally unimodular (where $\mathbf{I}$ is the $n \times n$ identity matrix).

Problem Formulation Using ILP - Real Estate Investment

We consider 6 buildings for investment.
The price and rental income for each of them are listed in the table.

building	1	2	3	4	5	6
price[mil. CZK]	5	7	4	3	4	6
income[thousand CZK]	16	22	12	8	11	19

Goal:

- maximize income

Constraints:

- investment budget is 14 mil CZK
- each building can be bought only once

Problem Formulation Using ILP - Real Estate Investment

We consider 6 buildings for investment.
The price and rental income for each of them are listed in the table.

building	1	2	3	4	5	6
price[mil. CZK]	5	7	4	3	4	6
income[thousand CZK]	16	22	12	8	11	19

Goal:

- maximize income

Constraints:

- investment budget is 14 mil CZK
- each building can be bought only once

Formulation

- $x_i = 1$ if we buy building i

$$
\begin{aligned}
\max \quad & z = 16x_1+22x_2+12x_3+ 8x_4+11x_5+19x_6 \\
s.t. \quad & 5x_1 + 7x_2 + 4x_3+ 3x_4 + 4x_5 + 6x_6 \leq 14 \\
& x_{i\in 1\cdots 6} \in \{0,1\}
\end{aligned}
$$

Adding Logical Formula $x_1 \Rightarrow x_2$

Another constraint:

- if building 1 is selected, then building 2 is selected too

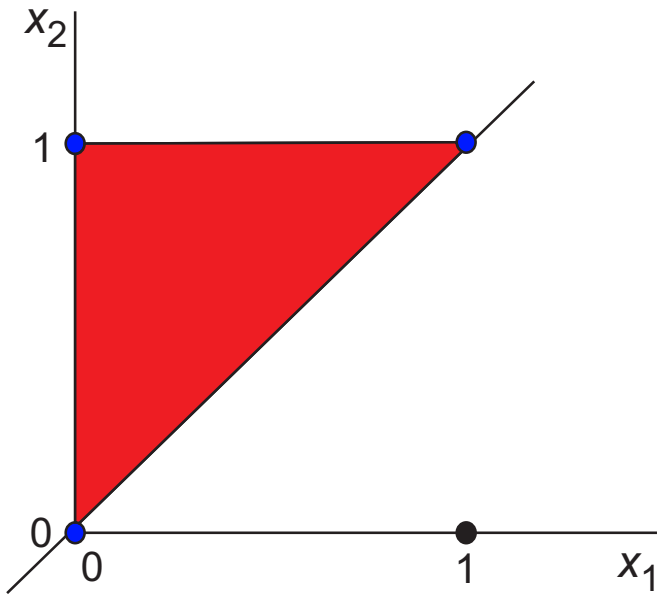
x_1	x_2	$x_1 \Rightarrow x_2$
0	0	1
0	1	1
1	0	0
1	1	1

Adding Logical Formula $x_1 \Rightarrow x_2$

Another constraint:

- if building 1 is selected, then building 2 is selected too

x_1	x_2	$x_1 \Rightarrow x_2$
0	0	1
0	1	1
1	0	0
1	1	1

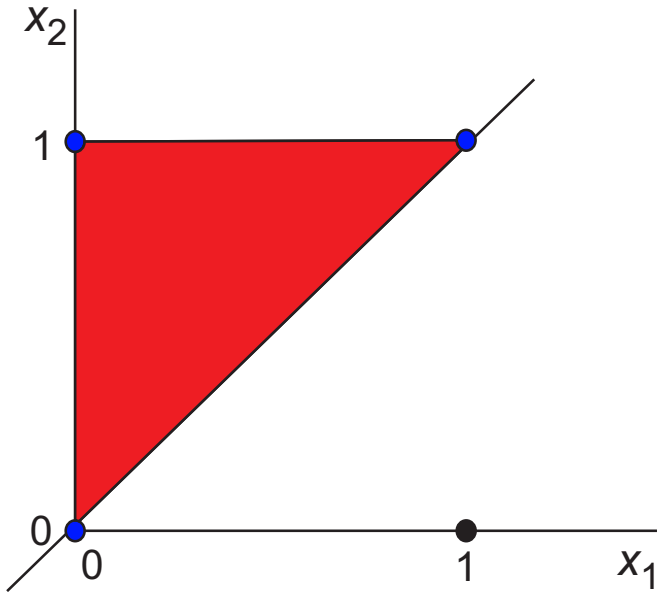


Adding Logical Formula $x_1 \Rightarrow x_2$

Another constraint:

- if building 1 is selected, then building 2 is selected too

x_1	x_2	$x_1 \Rightarrow x_2$
0	0	1
0	1	1
1	0	0
1	1	1



$$
\begin{aligned}
\max \quad & z = 16x_1+22x_2+12x_3+ 8x_4+11x_5+19x_6 \\
s.t. \quad & 5x_1 + 7x_2 + 4x_3+ 3x_4 + 4x_5 + 6x_6 \leq 14 \\
& x_2 \geq x_1 \\
& x_{i\in 1\cdots 6} \in \{0,1\}
\end{aligned}
$$

Adding Logical Formula $x_3 \Rightarrow \overline{x_4}$

Another constraint:

- If building 3 is selected, then building 4 is not selected.

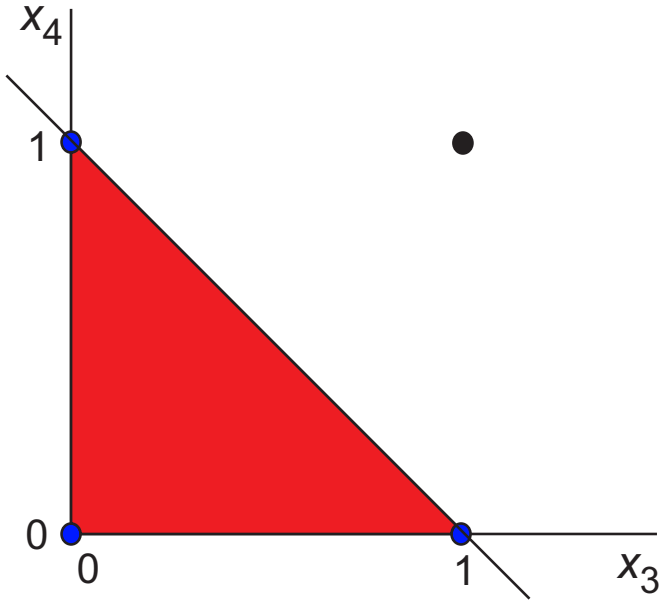
x_3	x_4	$x_3 \Rightarrow \overline{x_4}$
0	0	1
0	1	1
1	0	1
1	1	0

Adding Logical Formula $x_3 \Rightarrow \overline{x_4}$

Another constraint:

- If building 3 is selected, then building 4 is not selected.

x_3	x_4	$x_3 \Rightarrow \overline{x_4}$
0	0	1
0	1	1
1	0	1
1	1	0

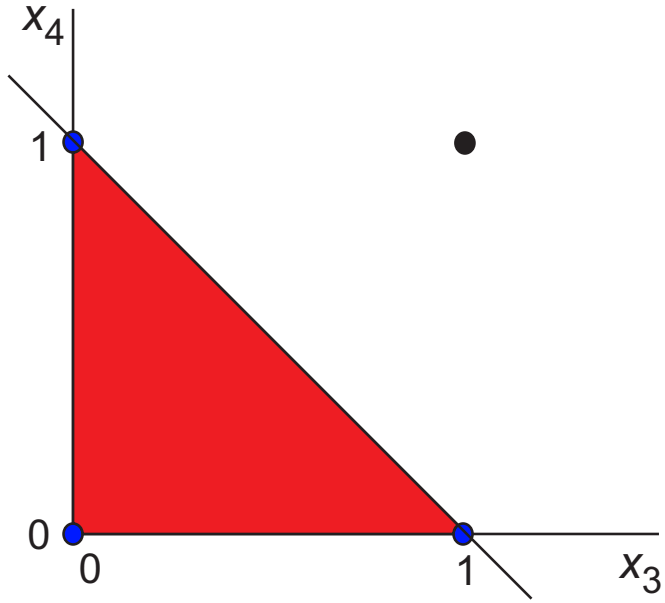


Adding Logical Formula $x_3 \Rightarrow \overline{x_4}$

Another constraint:

- If building 3 is selected, then building 4 is not selected.

x_3	x_4	$x_3 \Rightarrow \overline{x_4}$
0	0	1
0	1	1
1	0	1
1	1	0



$$
\begin{aligned}
\max \quad & z = 16x_1+22x_2+12x_3+8x_4+11x_5+19x_6 \\
s.t. \quad & 5x_1+7x_2+4x_3+3x_4+4x_5+6x_6 \leq 14 \\
& x_3+x_4 \leq 1 \\
& x_{i\in 1\cdots 6} \in \{0,1\}
\end{aligned}
$$

Adding Logical Formula x_5 XOR x_6

Another constraint:

- either building 5 is chosen or building 6 is chosen, but not both

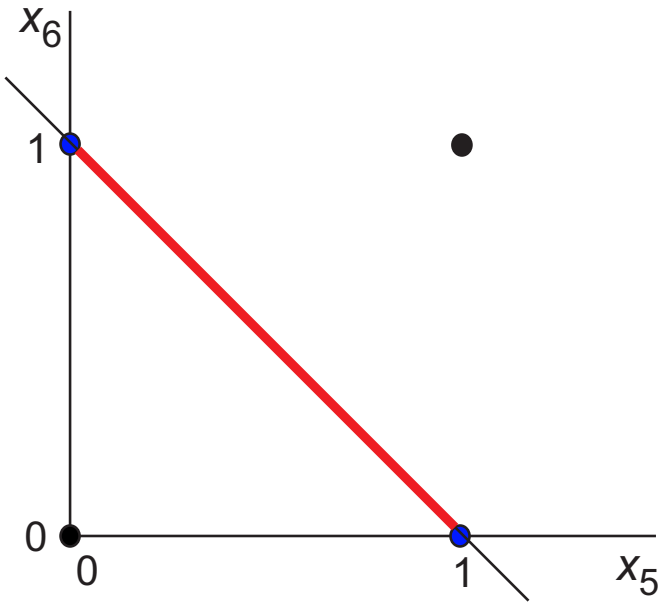
x_5	x_6	x_5 XOR x_6
0	0	0
0	1	1
1	0	1
1	1	0

Adding Logical Formula x_5 XOR x_6

Another constraint:

- either building 5 is chosen or building 6 is chosen, but not both

x_5	x_6	x_5 XOR x_6
0	0	0
0	1	1
1	0	1
1	1	0

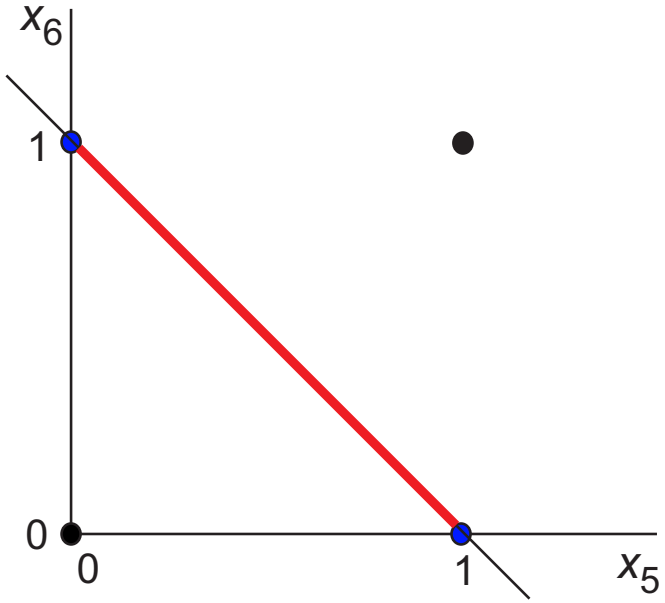


Adding Logical Formula x_5 XOR x_6

Another constraint:

- either building 5 is chosen or building 6 is chosen, but not both

x_5	x_6	x_5 XOR x_6
0	0	0
0	1	1
1	0	1
1	1	0



$$
\begin{aligned}
\max \quad & z = 16x_1+22x_2+12x_3+8x_4+11x_5+19x_6 \\
s.t. \quad & 5x_1+7x_2+4x_3+3x_4+4x_5+6x_6 \leq 14 \\
& x_5 + x_6 = 1 \\
& x_{i\in 1\cdots 6} \in \{0,1\}
\end{aligned}
$$

Adding Logical Formula - Homework

Formulate:

- building 1 must be chosen but building 2 can not
- at least 3 estates must be chosen
- exactly 3 estates must be chosen
- if estates 1 and 2 have been chosen, then estate 3 must be chosen too (x_1 AND x_2) $\Rightarrow x_3$
- exactly 2 estates can not be chosen

Problem Formulation Using ILP - Cloth Production

Labor demand, material demand and income per piece are:

product	T-shirt	shirt	robe	capacity	unit cost[CZK]
labor [person-hour]	3	2	6	160	1000
material [meter]	4	3	4	130	200
income [1000 CZK]	9.8	6.6	13.8		

Goal is to maximize the profit (i.e. total income minus total expenses)
Constraints:

- labor is on demand with maximum capacity of 160 person-hours
- material capacity is 130 meters

Problem Formulation Using ILP - Cloth Production

Labor demand, material demand and income per piece are:

product	T-shirt	shirt	robe	capacity	unit cost[CZK]
labor [person-hour]	3	2	6	160	1000
material [meter]	4	3	4	130	200
income [1000 CZK]	9.8	6.6	13.8		

Goal is to maximize the profit (i.e. total income minus total expenses)

Constraints:

- labor is on demand with maximum capacity of 160 person-hours
- material capacity is 130 meters

Formulation

- x_i is the amount of product i

$$
\begin{aligned}
\max \quad & z = 6x_1+4x_2+7x_3 = (9.8-3-0.8)x_1+(6.6-2-0.6)x_2+(13.8-6-0.8)x_3 \\
s.t. \quad & 3x_1+2x_2+6x_3 \leq 160 \\
& 4x_1+3x_2+4x_3 \leq 130 \\
x_{i\in 1\cdots 3} \quad & \in \mathbb{Z}_0^+
\end{aligned}
$$

Adding Relationship between Binary and Integer Variable

Another constraint:

- the fixed cost has to be covered to rent the machine

product	T-shirt	shirt	robe
machine cost	200 000	150 000	100 000

Adding Relationship between Binary and Integer Variable

Another constraint:

- the fixed cost has to be covered to rent the machine

product	T-shirt	shirt	robe
machine cost	200 000	150 000	100 000

Formulation

- add binary variable y_i such that $y_i = 1$ when the machine producing product i is the rent
- the objective function will be changed to
  $\max z = 6x_1 + 4x_2 + 7x_3 - 200y_1 - 150y_2 - 100y_3$
- join binary y_i with integer x_i

Adding Relationship between Binary and Integer Variable

Machine i is rented when product i is produced. We join binary y_i with integer x_i such that:

- $y_i = 0$ iff $x_i = 0$
- $y_i = 1$ iff $x_i \geq 1$

Adding Relationship between Binary and Integer Variable

Machine i is rented when product i is produced. We join binary y_i with integer x_i such that:

- $y_i = 0$ iff $x_i = 0$
- $y_i = 1$ iff $x_i \geq 1$

For range $x_i \in \langle 0, 100 \rangle$ these relations can be written as inequalities

- $x_i \leq 100 * y_i$
- $x_i \geq y_i$ ...we can omit this inequality due to the objective function ($x_i = 0, y_i = 1$ will not be chosen because we want x_i to be maximal and y_i minimal)

Considering Several Values

Another constraint:

- Total amount of work per week can be at most 40 or 80 or 120 or 160 hours depending on the number of Full Time Employees (FTEs)

Criterion adaptation:

- labor cost is proportional to the number of FTEs (one FTE costs 40)

We only want some values of person-hours to be available:

$$3x_1 + 2x_2 + 6x_3 \leq \text{ either 40 or 80 or 120 or 160}$$

Can be formulated using a set of additional variables $v_{i \in 1 \ldots 4} \in \{0, 1\}$ as:

$$3x_1 + 2x_2 + 6x_3 \leq 40v_1 + 80v_2 + 120v_3 + 160v_4$$
$$\sum_{i=1}^{4} v_i = 1$$

The objective function will be changed to:

$$\max z = (9.8 - 0.8)x_1 + (6.6 - 0.6)x_2 + (13.8 - 0.8)x_3 - 40v_1 - 80v_2 - 120v_3 - 160v_4$$

Note: due to the proportional labor cost, an alternative formulation could use $v \in \{1, 2, 3, 4\}$ as the number of FTEs

At least One of Two Constraints Must be Valid - OR

While modeling problems using ILP, we often need to express that the first, the second or both constraints hold. For example, $x_{i\in 1\ldots 4} \in \langle 0, 5\rangle$, $x_{i\in 1\ldots 4} \in \mathcal{R}$

$$\begin{array}{rc} \text{holds} & 2x_1 + 2x_2 \leq 8 \\ \text{or} & 2x_3 - 2x_4 \leq 2 \\ & \text{or both} \end{array}$$

This can be modeled by a big M, i.e. big positive number (here 15), and variable $y \in \{0, 1\}$ so it can "switch off" one of the inequalities.

$$\begin{aligned} 2x_1 + 2x_2 &\leq 8 + M \cdot y \\ 2x_3 - 2x_4 &\leq 2 + M \cdot (1 - y) \end{aligned}$$

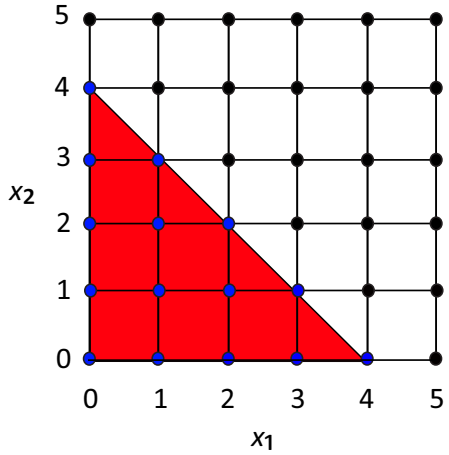
At least One of Two Constraints Must be Valid - OR

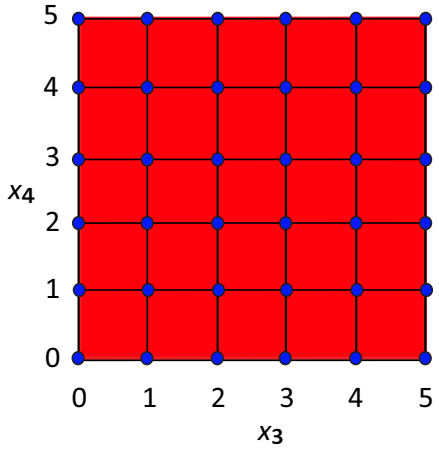
for $y = 0$ inequalities:

$$2x_1 + 2x_2 \leq 8 + M \cdot y$$
$$2x_3 - 2x_4 \leq 2 + M \cdot (1 - y)$$

reduce to:

$$2x_1 + 2x_2 \leq 8$$





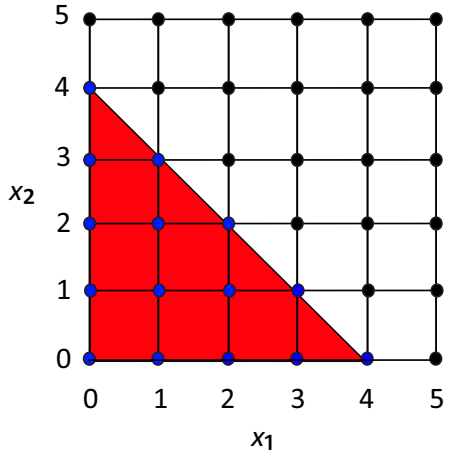
At least One of Two Constraints Must be Valid - OR

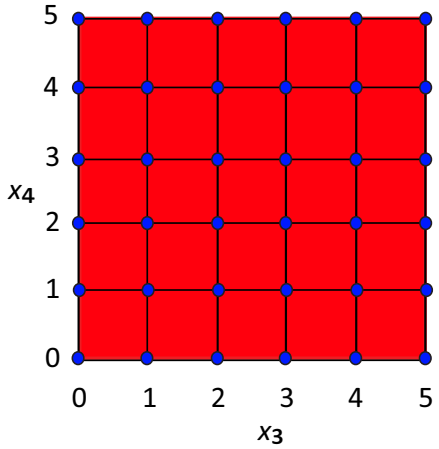
for $y = 0$ inequalities:

$$2x_1 + 2x_2 \leq 8 + M \cdot y$$
$$2x_3 - 2x_4 \leq 2 + M \cdot (1 - y)$$

reduce to:

$$2x_1 + 2x_2 \leq 8$$



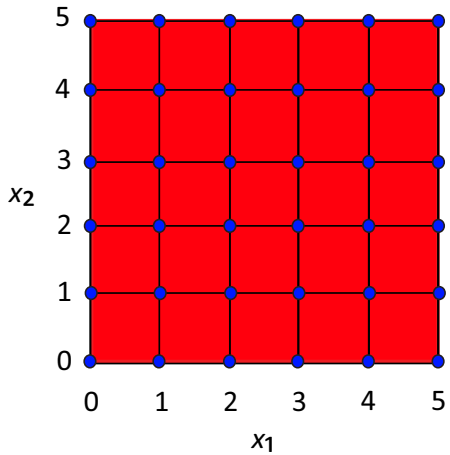


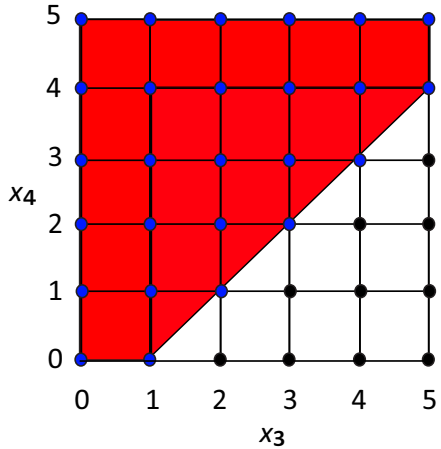
for $y = 1$ inequalities:

$$2x_1 + 2x_2 \leq 8 + M \cdot y$$
$$2x_3 - 2x_4 \leq 2 + M \cdot (1 - y)$$

reduce to:

$$2x_3 - 2x_4 \leq 2$$





At least One of Two Constraints Must be Valid - OR Homework

- To model OR relation between inequalities, draw the 2D solution space (x_1, x_2) of the system of inequalities with big M:

$$2x_1 + x_2 \leq 5 + M \cdot y$$
$$2x_1 - x_2 \leq 2 + M \cdot (1 - y)$$
$$y \in \{0, 1\}$$

- Example where OR overlaps with XOR:
  In 2D draw the solution space (x_1, x_2) of the system of inequalities. Note that the equations correspond to parallel lines. Is it possible to find x_1, x_2 such that both original inequalities $2x_1 + x_2 \leq 5$ and $2x_1 + x_2 \geq 10$ are valid simultaneously?

$$2x_1 + x_2 \leq 5 + M \cdot y$$
$$M \cdot (1 - y) + 2x_1 + x_2 \geq 10$$
$$y \in \{0, 1\}$$

At least One of Two Constraints Must be Valid Example: Non-preemptive Scheduling

$1 \left| r_j, \widetilde{d}_j \right| C_{max}$ ... NP-hard problem

- **Instance:** A set of non-preemptive tasks $\mathcal{T} = \{T_1, \ldots, T_i, \ldots T_n\}$ with release date r and deadline $\widetilde{d}$ should be executed on one processor. The processing times are given by vector p.
- **Goal:** Find a feasible schedule represented by start times s that minimizes completion time $C_{max} = \max_{i \in \langle 1,n \rangle} s_i + p_i$ or decide that it does not exist.

Example:

- T_i - chair to be produced by a joiner
- r_i - time, when the material is available
- $\widetilde{d}_i$ - time when the chair must be completed
- s_i - time when the chair production starts
- $s_i + p_i$ - time when the chair production ends

At least One of Two Constraints Must be Valid Example: Non-preemptive Scheduling

Since at the given moment, at most, one task is running on a given resource, therefore, for all task pairs T_i, T_j it must hold:

1. T_i precedes T_j $(s_j \geq s_i + p_i)$
2. or T_j precedes T_i $(s_i \geq s_j + p_j)$

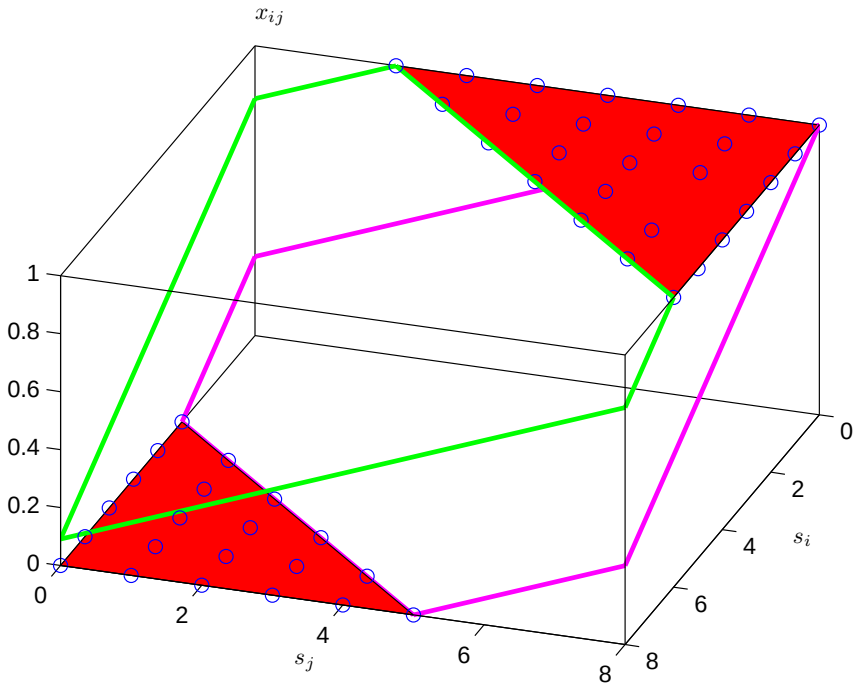
Note that (for $p_i > 0$) both inequalities can't hold simultaneously.
We need to formulate that at least one inequality holds. We will use variable $x_{ij} \in \{0, 1\}$ such that $x_{ij} = 1$ if T_i precedes T_j.
For every pair T_i, T_j we introduce inequalities:

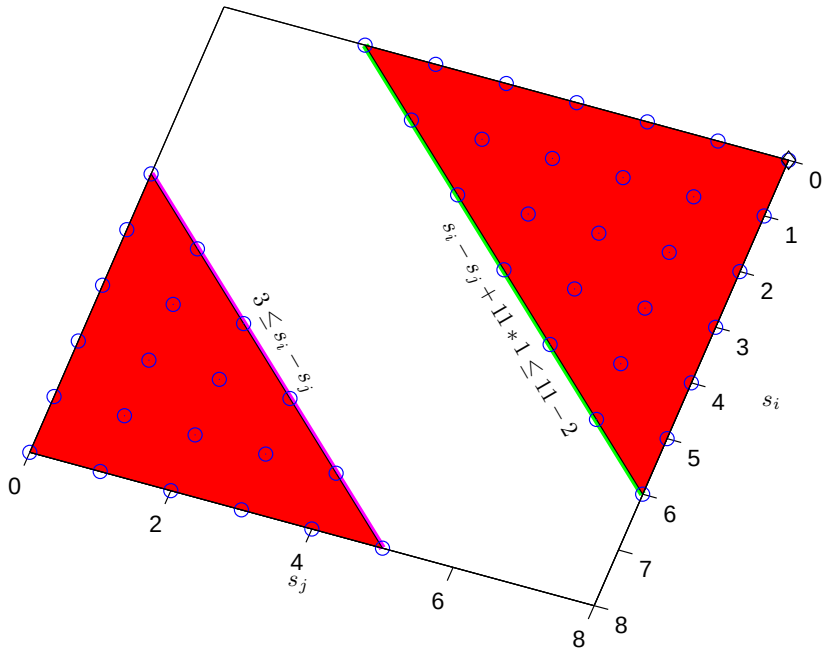
$$s_j + M \cdot (1 - x_{ij}) \geq s_i + p_i \quad \text{"switched off" when } x_{ij} = 0$$
$$s_i + M \cdot x_{ij} \geq s_j + p_j \quad \text{"switched off" when } x_{ij} = 1$$

Scheduling - Illustration of Non-convex Space

$$s_i \geq r_i \quad i \in 1..n \qquad \text{release date}$$
$$\widetilde{d}_i \geq s_i + p_i \quad i \in 1..n \qquad \text{deadline}$$
$$s_j + M \cdot (1 - x_{ij}) \geq s_i + p_i \quad i \in 1..n, j < i \qquad T_i \text{ precedes } T_j \text{ GREEN}$$
$$s_i + M \cdot x_{ij} \geq s_j + p_j \quad i \in 1..n, j < i \qquad T_j \text{ precedes } T_i \text{ VIOLET}$$

For example: $p_i = 2, p_j = 3, r_i = r_j = 0, \widetilde{d}_i = 10, \widetilde{d}_j = 11, M = 11$





Non-convex 2D space is a projection of two cuts of a 3D polytope (determined by the set of inequalities) in planes $x_{ij} = 0$ and $x_{ij} = 1$.

At least K of N Constraints Must Hold

We have N constraints and we need at least K of them to hold.
Constraints are of type:

$$\begin{aligned} f(x_1, x_2, \ldots, x_n) &\leq b_1 \\ f(x_1, x_2, \ldots, x_n) &\leq b_2 \\ &\vdots \\ f(x_1, x_2, \ldots, x_n) &\leq b_N \end{aligned}$$

Can be solved by introducing N variables $y_{i \in 1 \ldots N} \in \{0, 1\}$ such that

$$\begin{aligned} f(x_1, x_2, \ldots, x_n) &\leq b_1 + M \cdot y_1 \\ f(x_1, x_2, \ldots, x_n) &\leq b_2 + M \cdot y_2 \\ &\vdots \\ f(x_1, x_2, \ldots, x_n) &\leq b_N + M \cdot y_N \\ \textstyle\sum_{i=1}^{N} y_i &= N - K \end{aligned}$$

Note: If $K = 1$ and $N = 2$ we can use just scalar variable y and represent its negation as $(1 - y)$, see above slides with OR for details.

ILP Solvers

- CPLEX - proprietary IBM `http://www-03.ibm.com/software/products/en/ibmilogcpleoptistud`
- MOSEK - proprietary `http://www.mosek.com/`
- GLPK - free `http://www.gnu.org/software/glpk/`
- LP_SOLVE - free `http://groups.yahoo.com/group/lp_solve/`
- GUROBI - proprietary `http://www.gurobi.com/`

YALMIP - Matlab toolbox for modelling ILP problems
CVX - modeling framework

ILP - Conclusion

- NP-hard problem.
- Used to formulate majority of combinatorial problems.
- Often solved by branch and bound method.

References


Ravindra K. Ahuja, Thomas L. Magnanti, and James B. Orlin.
Network Flows: Theory, Algorithms, and Applications.
Prentice Hall, 1993.


B. H. Korte and Jens Vygen.
Combinatorial Optimization: Theory and Algorithms.
Springer, fourth edition, 2008.


James B. Orlin.
15.053 Optimization Methods in Management Science.
MIT OpenCourseWare, 2007.


Alexander Schrijver.
A Course in Combinatorial Optimization.
2006.